FestOS Backend Specification

Epinoia 2026 | Aarohan internal control portal | Python + FastAPI | Version 1

Save this file in the repo as `docs/BACKEND_SPEC.md`. It is the single source of truth the coding agent (opencode) reads before writing any code.

1. End goal

Build the backend of **FestOS**, an internal control portal for the core team of a three-day inter-college festival (about 2,400 expected participants, 38 colleges, 31 events, 9 venues, 14 food vendors, 48 core team members).

The backend must be more than CRUD. It must:

1. **Verify facts**: import messy CSVs, validate them, reconcile 2,836 raw participant records against ~2,400 expected, and never silently delete or fix data.
2. **Plan constrained resources**: produce defensible accommodation, scheduling and inventory plans that show violated and relaxed constraints and explain trade-offs.
3. **Act during disruptions**: given a disruption (late contingent, duplicate record, hostel closure, equipment failure, venue change, vendor shutdown, duplicate merch claim), preview its downstream impact, apply it to real system state, and allow undo.
4. **Explain every decision**: append-only audit log with before/after, actor, time and reason; mandatory reasons for high-risk overrides.
5. **Protect sensitive data**: role-based access, field masking for medical, accessibility, contact and room data, logged reveals.

Guiding principle: a change is complete only when its downstream effects are visible.

2. Non-goals

- No participant-facing registration site.
- No real personal data. Only the synthetic data pack.
- No frontend in this phase (a separate phase builds it against this API).
- No perfect optimiser. A defensible, explainable heuristic beats an opaque optimum.

3. Stack and conventions

Concern	Choice
Language	Python 3.11+
Framework	FastAPI
ORM	SQLAlchemy 2.x
Database	SQLite (file `festos.db`), single-file, easy reset
Validation	Pydantic v2
Data work	pandas, rapidfuzz (fuzzy duplicates)
Auth	JWT (python-jose or pyjwt), passlib/bcrypt for passwords
Optimisation	Greedy plus local search first. OR-Tools CP-SAT only as a later upgrade
Tests	pytest, httpx TestClient
Run	`uvicorn app.main:app --reload` and `python seed.py`

Conventions

- Engines (`engines/`) are **pure Python functions** that take plain data and return structured results. They never touch FastAPI. This makes them testable and reusable by the disruption engine.
- Every write path goes through a service function that calls `log_change()`. No endpoint writes to the database without an audit entry.
- Soft-delete everywhere (`deleted_at`). Keep `source_row_id` on imported entities.
- Rules and weights live in a config table or `config.py`, not hard-coded in logic, because a twist will arrive at hour 12.
- Three-state fields: `yes`, `no`, `unknown`. A blank value is always `unknown`, never `no`.
- All engine outputs are structured objects, never bare answers.

4. Project structure

```
festos/
  backend/
    app/
      main.py
      config.py            # weights, thresholds, rule toggles
      db.py                # engine, session, Base
      models.py            # all tables
      schemas.py           # pydantic models
      auth.py              # login, JWT, require_role, field masking
      audit.py             # log_change(), revert helpers
      routers/
        auth.py participants.py contingents.py accommodation.py
        events.py venues.py inventory.py vendors.py
        disruptions.py imports.py audit.py control.py assumptions.py
      engines/
        importer.py        # validate, quarantine, dedupe, reconcile
        allocator.py       # accommodation
        scheduler.py       # conflict detection and fix suggestions
        inventory.py       # ledger and stock derivation
        vendors.py         # coverage, compliance, settlement
        impact.py          # disruption preview/apply/undo
        attention.py       # control-center attention queue
    seed.py                # wipe DB, load CSVs, create users, reset demo
    tests/
  data/                    # raw CSVs, never edited
  docs/
    BACKEND_SPEC.md
    ASSUMPTIONS.md
  README.md
```

5. Data model

All tables have `id`, `created_at`, `updated_at`, `deleted_at`. Imported entities also keep `source_row_id` and `source_file`.

- **users** (username, password_hash, role, active)
- **participants** (raw_id, name, college, contingent_id, gender, housing_pref, accessibility_status [yes/no/unknown], accessibility_details, dietary_status [unknown/known], dietary_tags, medical_notes, phone, email, arrival_dt, departure_dt,

status [active/duplicate/cancelled/flagged/quarantined], merged_into, quality_flags JSON)

- **contingents** (name, college, size_declared, size_actual, leader_participant_id, escorts count, arrival_window, budget)
- **registrations** (participant_id, event_id, role [solo/member/captain], team_id, status)
- **teams** (event_id, name, contingent_id)
- **events** (name, venue_id, start_dt, end_dt, setup_min, teardown_min, rounds, capacity, equipment_needs JSON, judges_needed, volunteers_needed, status)
- **event_staff** (event_id, core_team_id, role [judge/volunteer/coordinator])
- **venues** (name, capacity, opens, closes, status [open/closed], location)
- **hostel_rooms** (hostel, floor, room_no, beds, accessible, gender_block, readiness [ready/cleaning/not_ready], cleaning_until, status [open/closed])
- **external_stays** (property, beds, cost_per_bed_night, travel_minutes, transport_available, safety_coverage, status)
- **allocations** (participant_id, room_id or external_stay_id, check_in, check_out, reason, relaxed_constraints JSON, cost, locked, plan_id, version, source [engine/manual/override])
- **allocation_plans** (preset, created_by, summary JSON, violations JSON, relaxations JSON, unplaced JSON, total_cost, status [draft/active/superseded])
- **inventory_items** (sku, name, category, size, kind [merch/logistics/equipment])
- **inventory_movements** (item_id, qty, from_location, to_location, status [available/reserved/in_transit/damaged/awaiting_return], at, reference, actor) *(ledger; stock is derived)*
- **merch_issues** (participant_id, canonical_participant_id, item_id, issued_by, issued_at; **unique** on (canonical_participant_id, item_id))
- **vendors** (name, menu JSON, dietary_coverage JSON, hygiene_cert_expiry, hours, capacity_per_day, rate, payment_terms, status [active/interrupted/suspended])
- **vendor_orders** (vendor_id, day, meal, qty, dietary_tag, amount, paid)
- **core_team** (name, role, shift_start, shift_end, skills, contact)
- **audit_log** (actor_id, action, entity_type, entity_id, before JSON, after JSON, reason, kind [manual/system/override/reveal], at)
- **import_issues** (file, row, field, severity [error/warning/info], code, message, status [open/resolved/accepted], resolution_note)
- **assumptions** (code like A-01, module, text, status, created_by)

- **disruptions** (type, payload JSON, impact_report JSON, status [previewed/applied/reverted], applied_by, applied_at, revert_snapshot JSON)
- **config_rules** (key, value JSON, description)

6. Features

12 modules, 46 features. Priority: **P0** must work in the live demo, **P1** differentiators, **P2** only if time remains.

Module 1: Auth, roles and privacy (P0)

1. Login with JWT, `/me` endpoint.
2. Roles: `director`, `accommodation_lead`, `events_lead`, `logistics_lead`, `vendor_lead`, `volunteer`, `viewer`.
3. `require_role` / `require_permission` dependency, server-side on every endpoint.
4. Field-level masking (medical, accessibility details, phone, email, room) based on role.
5. `POST /participants/{id}/reveal` returns unmasked fields and writes a `reveal` audit entry.
6. Seeded test credentials, one per role.

Module 2: Import, validation and reconciliation (P0)

7. Import all 11 CSVs with schema and type validation.
8. Cross-file integrity checks (registrations to missing participants, events to missing venues, contingent size mismatch, hostel bed totals vs 710, event end before start, events outside venue hours, expired hygiene certificates).
9. Quarantine bad rows into `import_issues`; never drop silently.
10. Duplicate detection (exact on key fields, fuzzy via rapidfuzz on name + college + phone/email), with `merged_into` links.
11. Merge and unmerge (reversible).
12. Reconciliation funnel endpoint: raw 2,836 → duplicates merged → cancelled/test/junk → flagged → active vs 2,400 expected.
13. Idempotent re-import and `seed.py` full reset.

Module 3: Participants and contingents (P0)

14. List with search (name, ID, college) and filters (flags, contingent, arrival day, status).

15. Unified 360° record: contingent, events, bed, merch, dietary, flags, audit history.
16. Uncertainty flags (arrival unknown, gender undeclared, dietary unverified, accessibility unknown).
17. Contingent summary (size declared vs actual, leader, escorts, split status).

Module 4: Accommodation (P0, core score)

18. Hard-constraint filter: accessibility, privacy, declared gender-segregation preference, capacity, readiness and cleaning, check-in/out windows. Blank is never read as consent or no-need.
19. Contingent-together placement with minimised splits and an explicit split reason.
20. Leader near group; faculty/escorts in separate but nearby rooms.
21. External stay spill-over by cheapest cost per bed, with transport and safety checks for late/early events and travel time.
22. Presets: `min_cost`, `max_cohesion`, `balanced`, with side-by-side comparison.
23. Plan object lists assignments, violations, relaxations, unplaced people, shortage (beds needed vs available), cost vs budget.
24. Manual-review queue for non-binary or unclear housing needs (no forced binary category).
25. Manual override requires a reason; high-risk overrides (accessibility relax, budget exceed) require `director` or `accommodation_lead`.
26. Lock allocations so re-runs do not move them.
27. Explain endpoint per placement (which rules fired and why).

Module 5: Events and scheduling (P0, core score)

28. Event and venue CRUD (with audit).
29. Conflict detection: participant, team, judge, volunteer, venue, equipment, venue capacity, operating hours, setup/teardown overlap, travel time between venues.
30. Suggested fixes (move slot, swap venue, reassign judge/volunteer) ranked by blast radius.
31. Reschedule/move event with live re-check and a conflict diff.

Module 6: Inventory and merchandise (P0)

32. Ledger-based movements with derived stock by location and status.
33. Transfers between locations with timestamps and references.

34. Demand vs stock shortfall by item, size and location.
35. Merch issue-once enforcement across duplicate participant clusters; returns `already issued at T by X` on repeat.
36. Equipment failure marking and substitute suggestion.

Module 7: Vendors (P0)

37. Dietary coverage matrix (participant needs per day vs vendor menu tags), uncovered categories highlighted.
38. Compliance status (expired or missing hygiene cert blocks assignment).
39. Interruption handling with reroute suggestions (vendors with spare capacity and matching tags).
40. Settlement (rate × quantity, payment terms, outstanding).

Module 8: Disruption engine (P0, decides the live test)

41. Seven handlers, each with `preview` and `apply`: late contingent, duplicate record, hostel closure, equipment failure, venue change, vendor shutdown, duplicate merch claim.
42. Impact report schema: what changed, who is affected (counts and names), constraints now violated, suggested actions, cost delta.
43. Undo/revert from the stored snapshot, audited.

Module 9: Control center (P0)

44. Attention queue ranked by severity across all modules, with counts for unplaced people, conflicts, stock risks, vendor risks, open import issues.

Module 10: Audit (P0)

45. Append-only log, filter by actor, entity, kind, date; override entries highlighted.

Module 11: Assumptions register (P1)

46. CRUD for assumptions (`A-01`, ...) plus a per-module data-quality score.

Module 12: Extras (P2)

Forecasts (arrival load, meals per vendor per day, bed shortage), CSV/JSON export of plans, QR payloads for check-in and merch, volunteer "my tasks" endpoint.

7. Engine contracts

Every engine returns a structured result. Example for the allocator:

```
{
  "plan_id": 12,
  "preset": "balanced",
  "assignments": [{"participant_id": 1, "room_id": 4, "reason": "...",
"relaxed": []}],
  "violations": [{"code": "ACCESSIBILITY_UNMET", "participant_id": 9,
"detail": "..."}],
  "relaxations": [{"code": "CONTINGENT_SPLIT", "contingent_id": 3, "parts":
2, "reason": "..."}],
  "unplaced": [{"participant_id": 77, "reason": "no accessible bed
available"}],
  "shortage": {"beds_needed": 2500, "beds_available": 1450},
  "cost": {"external_total": 123400, "budget": 150000},
  "explanations": {}
}
```

Rules that must never be broken silently: accessibility needs, declared privacy needs, declared gender-segregation preference, room capacity, readiness/cleaning windows. If they cannot be met the person goes into `unplaced` or `violations`, never quietly traded for a better score.

Disruption impact report:

```
{
  "disruption_id": 5, "type": "hostel_closure", "mode": "preview",
  "state_changes": [...], "affected": {"participants": 48, "events": 2,
"vendors": 0},
  "new_violations": [...], "suggested_actions": [...], "cost_delta": 18000
}
```

Impact is computed by reusing the engines (hostel closure calls the allocator on displaced people only, with a minimise-moves penalty and locked assignments respected; venue change calls the scheduler; vendor shutdown calls the vendor engine).

8. API surface (about 60 endpoints)

Group	Endpoints
Auth	`POST /auth/login`, `GET /auth/me`
Import	`POST /imports/upload`, `GET /imports/issues`, `PATCH /imports/issues/{id}`, `GET /imports/reconciliation`, `POST /imports/reset`
Participants	`GET /participants`, `GET /participants/{id}`, `POST /participants/{id}/reveal`, `PATCH /participants/{id}`, `POST /participants/merge`, `POST /participants/{id}/unmerge`, `GET /participants/duplicates`
Contingents	`GET /contingents`, `GET /contingents/{id}`
Accommodation	`POST /accommodation/plans` (preset), `GET /accommodation/plans`, `GET /accommodation/plans/{id}`, `GET /accommodation/compare`, `POST /accommodation/plans/{id}/activate`, `POST /accommodation/override`, `POST /accommodation/lock`, `GET /accommodation/explain/{allocation_id}`, `GET /accommodation/shortage`, `GET /accommodation/review-queue`
Events	`GET/POST/PATCH /events`, `GET /venues`, `GET /schedule/conflicts`, `GET /schedule/suggestions/{conflict_id}`, `POST /schedule/move`
Inventory	`GET /inventory/stock`, `POST /inventory/movements`, `GET /inventory/shortfalls`, `POST /merch/issue`, `GET /merch/issues`

Group	Endpoints
Vendors	`GET /vendors`, `GET /vendors/coverage`, `GET /vendors/compliance`, `POST /vendors/{id}/interrupt`, `GET /vendors/settlement`
Disruptions	`GET /disruptions/types`, `POST /disruptions/preview`, `POST /disruptions/{id}/apply`, `POST /disruptions/{id}/revert`, `GET /disruptions`
Control	`GET /control/attention`, `GET /control/kpis`
Audit and assumptions	`GET /audit`, `GET/POST/PATCH /assumptions`, `GET /quality`

9. Permission matrix

Capability	director	accommodation	events	logistics	vendor	volunteer	viewer
View masked participant	yes	yes	yes	yes	yes	yes	yes
Reveal sensitive fields	yes	yes (room, accessibility)	no	no	dietary only	no	no
Run/activate allocation	yes	yes	no	no	no	no	no
High-risk override	yes	yes (with reason)	no	no	no	no	no
Edit events/schedule	yes	no	yes	no	no	no	no
Inventory movements	yes	no	no	yes	no	limited	no
Merch issue	yes	no	no	yes	no	yes	no
Vendor actions	yes	no	no	no	yes	no	no
Apply disruption	yes	scoped	scoped	scoped	scoped	no	no
View audit	yes	own	own	own	own	no	yes

10. Build phases and acceptance criteria

Build in this order. Do not start a phase until the previous one passes its checks.

Phase 0: Skeleton (30 min). Folder structure, `requirements.txt`, `main.py` with health check, `db.py`, `config.py`. *Check:* `uvicorn` starts and `/health` returns ok.

Phase 1: Models, audit, auth (1.5 h). Full `models.py`, `audit.log_change()`, JWT login, `require_role`, seeded users. *Check:* login works, a protected endpoint returns 403 for the wrong role, an audit row is written on a test write.

Phase 2: Import and reconciliation (2 h). Importer, validator, quarantine, dedupe, funnel, `seed.py`. *Check:* `python seed.py` rebuilds the DB from scratch; `/imports/reconciliation` shows 2,836 → N with every dropped or merged record accounted for.

Phase 3: Participants and contingents (1 h). List, search, 360°, masking, reveal. *Check:* masked fields hidden for `volunteer`; `reveal` logs an audit entry.

Phase 4: Engines (4 h). Scheduler, inventory ledger, vendors, then allocator (greedy first). Pure functions with pytest. *Check:* each engine has tests; allocator never places accessibility-need participants in inaccessible beds.

Phase 5: Disruption engine (2 h). `preview`, `apply`, `revert` for all seven types, snapshots. *Check:* each type changes real DB state and the impact report matches.

Phase 6: Control center and audit endpoints (1 h). Attention queue, KPIs, audit filters.

Phase 7: Hardening (1 h). Input validation, error handling, README with setup and credentials, assumptions file, final seed determinism.

11. Testing requirements

- Pytest for every engine, with fixture data and the real pack.
- One test per hard constraint (accessibility, privacy, gender preference, capacity, readiness, windows).
- One test per disruption type covering preview, apply and revert.
- Merch issue-once test including a duplicate-participant cluster.
- Permission tests (each role against 3 representative endpoints).
- Idempotency test: seed twice, same result.

12. Definition of done (backend)

- [ ] `python seed.py && uvicorn app.main:app` gives a working API with demo data
- [ ] OpenAPI docs at `/docs` list every endpoint above
- [ ] 2,836 → N reconciliation is explained and queryable
- [ ] A sample accommodation plan, schedule conflict report and stock report can be generated from the pack
- [ ] All seven disruptions preview, apply and revert, and change real state
- [ ] Every write is in the audit log; overrides require a reason
- [ ] Sensitive fields are masked by role; reveals are logged
- [ ] `docs/ASSUMPTIONS.md` is populated
- [ ] README has setup steps, test credentials, known limitations and external dependencies

13. Rules for the coding agent

1. Read `data/README.txt` and profile every CSV before writing the importer. Do not guess column names.
2. Never delete or silently correct source data. Quarantine, flag and link.
3. Never treat a blank as `no`, consent, eligibility, gender or dietary clearance.
4. Keep engines pure and testable. Keep rules and weights in config.
5. Every state change writes to `audit_log`.
6. Record every assumption you make in `docs/ASSUMPTIONS.md` as `A-NN` with the module and reasoning.
7. After each phase, run the tests, report what passed, and stop for review before the next phase.
8. Prefer small, working vertical slices over broad unfinished code.